

Web Semântica

Proposta / Implementação / Avaliação Preliminar

Pedro Saito

Universidade Federal do Rio de Janeiro

2026-06-23



1. Proposta

1.1 Objetivo

*Construir uma engine mínima para consultas federadas em SPARQL,
mensurável pelo FedShop e implementada em Go.*

Benchmark

Reimplementar o pipeline do FedShop em Python, removendo a dependência central do Snakemake.

CONCLUÍDO

Protótipo

Validar parser, seleção de fontes, execução remota e artefatos antes da implementação definitiva.

CONCLUÍDO

Engine em Go

Consolidar as estratégias do WS1 em uma arquitetura pequena, testável e integrada ao benchmark.

**EM
DESENVOLVIMENTO**

FedShop

Benchmark para medir escalabilidade de engines SPARQL federadas.

Cada execução fixa uma combinação:

engine × **consulta** × **instância** × **batch** × **tentativa**

O objetivo é isolar o impacto do tamanho da federação e da estratégia da engine.

Eixo	Variações consideradas
Consultas	12 templates (q01–q12)
Instâncias	10 valores concretos por template
Escala	10 batches: 20, 40, ..., 200 membros
Fontes	vendedores + sites de avaliação
Tentativas	repetições controladas por combinação
Engines	FedX, CostFed, FedUP/RSA, protótipos

1.2 Ambiente Experimental

1. Proposta

Métricas: **tempo**, **HTTP**, **transferência**, **fontes selecionadas** e **timeout**.

2. Implementação

2.1 fedshop-py: Pipeline Reprodutível

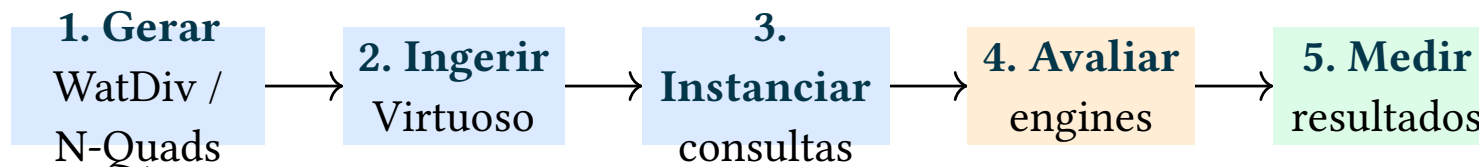


Figura 1: *Fluxo reimplementado como pacote e CLI Python.*

Mudanças principais

- configuração YAML tipada;
- manipulação SPARQL com rdflib;
- execução via adapters;
- métricas agregadas com pandas.

Artefatos preservados

- `results.csv` e `provenance.csv`;
- `source_selection.txt`;
- `query_plan.txt`;
- tempos, requisições e transferência.

2.2 pyfedx: Protótipo de Validação

Papel do protótipo

Reduzir incertezas antes da engine em Go: exercitar o caminho completo entre uma consulta SPARQL e os endpoints remotos.

Escopo implementado

- BGPs, FILTER, OPTIONAL e UNION;
- ORDER BY, LIMIT e DISTINCT;
- seleção por ASK com cache;
- joins de bindings compatíveis;
- execução HTTP e saída FedShop.

```
parse(query)
```

↓

```
select_sources(ASK + cache)
```

↓

```
execute(SERVICE requests)
```

↓

```
join + filter + project
```

↓

```
results / sources / stats / plan
```

text

Protótipo de pesquisa — não é a engine final.

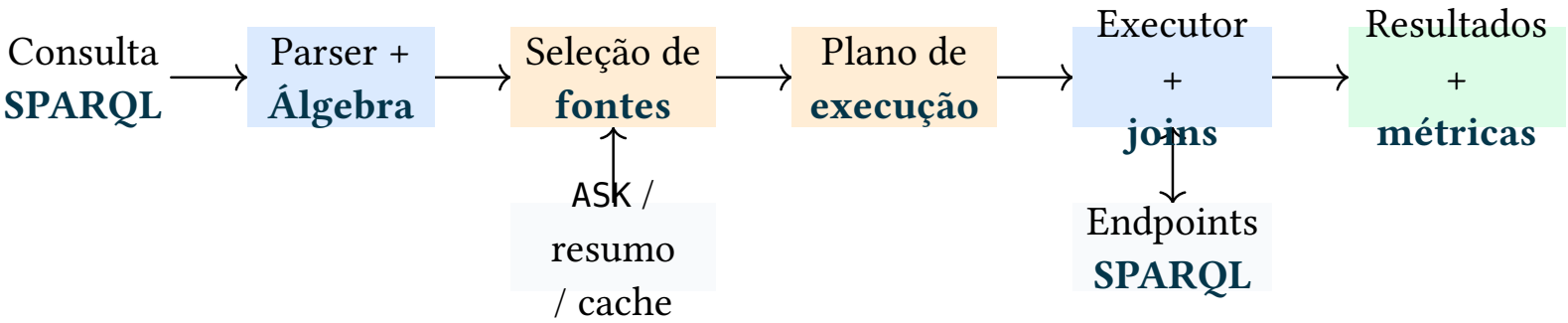


Figura 2: Arquitetura incremental da implementação definitiva.

Referência do WS1	Incorporação planejada	Estado
FedX	ASK, cache, grupos exclusivos e bound joins	EM DESENVOLVIMENTO
SPLENDID / CostFed	resumos, cardinalidades e ordenação	PRÓXIMO
ANAPSID	timeouts e continuidade sob falhas	PRÓXIMO

3. Avaliação

Dimensão	Métricas	Pergunta respondida
Correção	nb_results, mismatch	A resposta é equivalente à referência?
Tempo	source_selection, planning, exec_time	Onde o tempo é consumido?
Rede	ask, http_req, data_transfer	Quanto custa consultar a federação?
Seleção	TPWSS, RWSS, fontes distintas	Quantas fontes irrelevantes foram escolhidas?
Robustez	timeout, erro de runtime	O plano continua útil sob falhas?

Variante	ASK	Cache	Bound	Metadados
Baseline	—	—	—	—
Seleção dinâmica	✓	—	—	—
Seleção reutilizada	✓	✓	—	—
Execução vinculada	✓	✓	✓	—
Plano completo	✓	✓	✓	✓

Controle experimental

Mesma consulta, instância, batch, tentativa e infraestrutura para todas as variantes.

Efeito esperado

Comparar cada componente por tempo, requisições, transferência e precisão da seleção.

69

testes do fedshop-py

Configuração, álgebra, geração, ingestão, avaliação, métricas e adapters.

24

testes internos do pyfedx

Parser, filtros, joins, OPTIONAL, UNION, ordenação, seleção e execução simulada.

Os testes validam comportamento e integração local.
Ainda não representam desempenho no workload completo do FedShop.

PLACEHOLDER — medições serão inseridas antes da apresentação

Engine	Tempo	HTTP	TPWSS	Status
FedX	—	—	—	pendente
CostFed	—	—	—	pendente
pyfedx	—	—	—	pendente
Engine Go	—	—	—	em desenvolvimento

Resultados só serão comparados após validar equivalência com o conjunto de referência.

1

Smoke test

Configuração pequena, uma consulta e uma instância. Validar todo o caminho e os artefatos.

2

Execução filtrada

Fixar engine, consulta, batch e tentativa.
Comparar resultados e diagnosticar falhas.

3

Workload completo

Executar 12 templates, 10 instâncias e até 200 membros, com repetição controlada.

Correção → estabilidade → escala → comparação

```
# Preparar os dados
fedshop generate products
fedshop generate sources
fedshop ingest batch 0

# Instanciar consultas e avaliar
fedshop query run-all --batch-id 0
fedshop evaluate run-all \
  --engine pyfedx --query q01
```

bash

Saída por execução

```
results.csv
provenance.csv
source_selection.txt
query_plan.txt
stats.csv
```

Os mesmos contratos serão implementados pelo adapter da engine em Go.

Ambiente atual

Linguagem	Python 3.12
CLI	click + uv
SPARQL	rdflib / SPARQLWrapper
Dados	WatDiv + Virtuoso
Engine final	Go

Sequência de entrega

1. Completar o núcleo da engine em Go.
2. Implementar seleção ASK e cache.
3. Adicionar joins e execução SERVICE.
4. Criar adapter compatível com fedshop-py.
5. Rodar smoke, ablação e benchmark completo.
6. Inserir resultados no slide preliminar.

Próximo marco: uma consulta FedShop completa executada pela engine Go, com resultados, fontes, plano e métricas reproduzíveis.

Referências

- (1) Acosta, M.; Vidal, M.-E.; Lampo, T.; Castillo, J.; Ruckhaus, E. ANAPSID: An Adaptive Query Processing Engine for SPARQL Endpoints. Em *The Semantic Web – ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23–27, 2011, Proceedings, Part I*; Lecture Notes in Computer Science; Springer, 2011; Vol. 7031, pp 18–34. https://doi.org/10.1007/978-3-642-25073-6_2.
- (2) Dang, M.-H.; Aimonier-Davat, J.; Molli, P.; Hartig, O.; Skaf-Molli, H.; Crom, Y. L. FedShop: A Benchmark for Testing the Scalability of SPARQL Federation Engines. Em *The Semantic Web – ISWC 2023: 22nd International Semantic Web Conference, Athens, Greece, November 6–10, 2023, Proceedings, Part II*; Lecture Notes in Computer Science; Springer, 2023; Vol. 14266, pp 285–301. https://doi.org/10.1007/978-3-031-47243-5_16.
- (3) Saleem, M.; Potocki, A.; Soru, T.; Hartig, O.; Ngomo, A.-C. N. CostFed: Cost-Based Query Optimization for SPARQL Endpoint Federation. Em *Proceedings of the 14th International Conference on Semantic Systems (SEMANTiCS 2018)*; Procedia Computer Science; Elsevier, 2018; Vol. 137, pp 163–174. <https://doi.org/10.1016/j.procs.2018.09.016>.
- (4) Schwarte, A.; Haase, P.; Hose, K.; Schenkel, R.; Schmidt, M. FedX: Optimization Techniques for Federated Query Processing on Linked Data. Em *The Semantic Web – ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23–27, 2011, Proceedings, Part I*; Lecture Notes in Computer Science; Springer, 2011; Vol. 7031, pp 601–616. https://doi.org/10.1007/978-3-642-25073-6_38.
- (5) Görlitz, O.; Staab, S. SPLENDID: SPARQL Endpoint Federation Exploiting VOID Descriptions. Em *Proceedings of the Second International Workshop on Consuming Linked Data (COLD 2011)*; CEUR Workshop Proceedings; CEUR-WS.org, 2011; Vol. 782, pp 13–24.
- (6) Charalambidis, A.; Troumpoukis, A.; Konstantopoulos, S. SemaGrow: Optimizing Federated SPARQL Queries. Em *Proceedings of the 11th International Conference on Semantic Systems (SEMANTiCS 2015)*; ACM, 2015; pp 121–128. <https://doi.org/10.1145/2814864.2814886>.